implement an 8-puzzle problem using i)DFS

```
def print_as_grid(state):
    for i in range(0, 9, 3):
        print(state[i:i+3])


def dfs(state, goal, visited, path):
    if state == goal:
        print("\nGoal Found!")

        for step, board in enumerate(path):
            print("\nStep", step)
            print_as_grid(board)

        return True

    visited.add(state)

    zero_index = state.index(0)
    zero_row = zero_index // 3

    # Down, Right, Up, Left
    for move in [3, 1, -3, -1]:
        new_pos = zero_index + move

        if 0 <= new_pos < 9:
            new_row = new_pos // 3

            # Prevent moving left or right across rows
            if move in [3, -3] or zero_row == new_row:
                new_state = list(state)

                new_state[zero_index], new_state[new_pos] = (
                    new_state[new_pos], new_state[zero_index]
                )

                new_state = tuple(new_state)

                if new_state not in visited:
                    path.append(new_state)

                    if dfs(new_state, goal, visited, path):
                        return True

                    path.pop()
```

```
    return False


print("Enter initial state (0 for blank):")
state = tuple(map(int, input().split()))

print("Enter goal state:")
goal = tuple(map(int, input().split()))

if (len(state) != 9 or len(goal) != 9
        or set(state) != set(range(9))
        or set(goal) != set(range(9))):
    print("Invalid input! Enter numbers 0 to 8 exactly once.")

else:
    print("\nInitial State:")
    print_as_grid(state)

    visited = set()
    path = [state]

    if not dfs(state, goal, visited, path):
        print("Goal not found!")
```

output:

```
Enter initial state (0 for blank):
1 2 3 4 0 6 7 5 8
Enter goal state:
1 2 3 4 5 6 7 8 0

Initial State:
(1, 2, 3)
(4, 0, 6)
(7, 5, 8)

Goal Found!

Step 0
(1, 2, 3)
(4, 0, 6)
(7, 5, 8)

Step 1
(1, 2, 3)
(4, 5, 6)
(7, 0, 8)

Step 2
(1, 2, 3)
(4, 5, 6)
(7, 8, 0)
```

#implement an 8-puzzle problem using
ii)Iterative deepening search(IDS) algorithm

```
def print_as_grid(state):
    print("[", state[0], state[1], state[2], "]")
    print("[", state[3], state[4], state[5], "]")
    print("[", state[6], state[7], state[8], "]")
    print("-----------")


def dfs_limited(state, goal, limit, depth, visited, path):

    if state == goal:
        print("\n--- Goal Reached! ---")

        for step, board in enumerate(path):
            print("\nStep", step)
            print_as_grid(board)

        return True

    if depth >= limit:
        return False

    visited.add(state)

    zero = state.index(0)
    zero_row = zero // 3
    moves = [-3, +3, -1, +1]

    for move in moves:
        new_pos = zero + move

        if 0 <= new_pos <= 8:
            new_row = new_pos // 3

            if move in [-1, +1] and zero_row != new_row:
                continue

            new_state_list = list(state)
            new_state_list[zero], new_state_list[new_pos] = (
                new_state_list[new_pos], new_state_list[zero]
            )
```

```
            new_state = tuple(new_state_list)

            if new_state not in visited:
                path.append(new_state)

                if dfs_limited(new_state, goal, limit,
                               depth + 1, visited, path):
                    return True

                path.pop()

    visited.remove(state)
    return False


def iterative_deepening_search(initial_state, goal):
    depth_limit = 0

    while depth_limit <= 30:
        print("\nSearching at depth limit:", depth_limit)
        visited = set()
        path = [initial_state]

        if dfs_limited(initial_state, goal, depth_limit,
                       0, visited, path):
            print("\nSolution found successfully at depth:",
                  depth_limit)
            return True

        depth_limit += 1

    print("No solution found within constraints.")
    return False


# Take user input
print("Enter initial state (0 for blank):")
initial_state = tuple(map(int, input().split()))

print("Enter goal state (0 for blank):")
goal_state = tuple(map(int, input().split()))

# Validate input
if (len(initial_state) != 9 or len(goal_state) != 9
        or set(initial_state) != set(range(9))
        or set(goal_state) != set(range(9))):
    print("Invalid input! Enter numbers 0 to 8 exactly once.")

else:
```

```
print("\nInitial State:")
print_as_grid(initial_state)

print("\n--- Running Iterative Deepening Search ---")
iterative_deepening_search(initial_state, goal_state)
```

output:

```
Enter initial state (0 for blank):
1 2 3 4 0 6 7 5 8
Enter goal state (0 for blank):
1 2 3 4 5 6 7 8 0

Initial State:
[ 1 2 3 ]
[ 4 0 6 ]
[ 7 5 8 ]
-----------

--- Running Iterative Deepening Search ---

Searching at depth limit: 0

Searching at depth limit: 1

Searching at depth limit: 2

--- Goal Reached! ---

Step 0
[ 1 2 3 ]
[ 4 0 6 ]
[ 7 5 8 ]
-----------

Step 1
[ 1 2 3 ]
[ 4 5 6 ]
[ 7 0 8 ]
-----------

Step 2
[ 1 2 3 ]
[ 4 5 6 ]
[ 7 8 0 ]
-----------

Solution found successfully at depth: 2
```